AI Portal — The Enterprise AI Orchestration Platform

Implementation Whitepaper — Draft v0.3

Subtitle: An AI-native orchestration platform that turns business documents into production-ready, standards-compliant software — by letting any IDE, agent, or developer tap into AD Ports' shared fabric of MCPs, agents, skills, specifications, and architectural standards.

Document status: Draft for review **Audience:** CTO, Heads of Engineering, Architects, Technology Leadership, Platform Engineering **Classification:** Internal / Confidential

---

Table of Contents

1. Executive Summary
2. Vision & Positioning
3. The Problem We Are Solving
4. Platform Architecture Overview
5. The AI Portal — Central Control Plane
6. The Capability Fabric — MCPs, APIs, CLIs, Skills, Specs, Instructions
7. IDE & Agent Integration (Copilot, Cursor, Claude Code, Terminal)
8. The Orchestration Agent & Agent Hierarchy
9. Specialized Delivery Agents
10. Worked Example — "Generate a New JUL-Style Microservice Application"
11. Guardrails, Hooks & Role-Based Governance
12. Artifact Generation, Versioning & Shared Project Context
13. The Immutable Pipeline Ledger — Recording Every Stage, Artifact, and Approval
14. Review & Approval Workflow
15. Out-of-the-Box Shared Services (Identity, MDM, Notifications, QA Framework)
16. QA Automation Agent — Playwright, Load, Accessibility & Visual Regression
17. Deployed-System Verification — Postman-Style Integration Testing
18. Security Framework Integration — SonarQube, Checkmarx & the Secure Pipeline
19. Framework Migration Guidelines Engine
20. Fleet-Wide Upgrade & Centralized Maintenance Agent
21. BA & PM Automation — User Story Generation for Jira & Azure DevOps

22. Auto-Implementation from Jira Tickets
23. Continuous Vulnerability & Framework-Obsolescence Radar
24. Live Health Monitoring of AD Ports Services
25. Pull Request Review Agent & Code Quality Gate
26. Project Registry & Cross-Project Component Reuse
27. Legacy System Migration Mode
28. AD Ports Ecosystem Agents (MPay, CRM, CMS, ERP, JUL, PCS)
29. Additional Capabilities Worth Adding
30. Technology Stack & Build-vs-Integrate Decisions
31. Phased Delivery Approach
32. Risk Analysis & Mitigation
33. Success Metrics & KPIs
34. Conclusion & Next Steps

---

1. Executive Summary

AD Ports Group builds large, integration-heavy enterprise systems — JUL (Angola National Single Window), PCS (Port Community System), Mirsal/customs integrations, internal platforms for MPay, CRM, CMS, and ERP. Every new project repeats the same weeks of bootstrap work: scaffolding a .NET/CQRS backend, wiring Keycloak, setting up PostgreSQL with DbUp or EF Core migrations, standing up an Angular micro-frontend, writing GitLab CI pipelines, Helm charts for AKS, Terraform/Pulumi for Azure infrastructure, observability, notification plumbing, and integration adapters for MPay, CRM, and the other shared systems. Worse, once projects are live, maintenance work — framework upgrades, security patches, library updates — has to be repeated separately in every project, consuming engineering capacity disproportionate to the business value delivered.

This whitepaper proposes the **AI Portal** — a centralized orchestration platform that eliminates both the one-time bootstrap and the ongoing maintenance repetition. The AI Portal is **not** another code generator or low-code platform. It is a **fabric of MCPs, agents, skills, specifications, and architectural standards** that any AI coding tool — Copilot, Cursor, Claude Code, a terminal agent, or a developer's own script — can connect to and leverage.

The proposition has four parts.

Generate. A solution architect feeds a BRD, HLD, and ICD into the Portal. The Orchestration Agent reads the documents, consults skill, spec, and instruction agents for AD Ports' architectural standards, proposes an architecture with draw.io diagrams and a Docusaurus specification site, and — once approved by the technical lead — delegates to specialized sub-agents that create Azure DevOps or GitLab repositories, scaffold the .NET/CQRS backend, generate the Angular or React micro-frontend, wire integrations to MPay/CRM/Keycloak, build CI/CD pipelines with SAST and SCA baked in, provision Azure infrastructure, generate Playwright automation suites, produce Postman-style integration collections for deployed-system verification, and return repository URLs along with live staging URLs.

Maintain. Every generated project is registered in a central **Project Registry**. A Fleet-Wide Upgrade Agent orchestrates maintenance campaigns — "upgrade every project from .NET 6 to .NET 10" becomes a single coordinated campaign across the entire portfolio, not 40 separate manual upgrades. Security patches propagate the same way. Each campaign respects per-project readiness, runs the project's full QA suite, and produces a consolidated verification report.

Operate. A live **Service Health Monitoring** layer tracks every running AD Ports service with configurable health checks, surfaces status in an admin UI, and fires notifications through Teams, Slack, email, or PagerDuty when services degrade or fail.

Govern. Every decision the Portal ever makes is recorded in an immutable **Pipeline Ledger** — every artifact version, every approval, every comment, every delegation, every deployment — cryptographically chained and permanently queryable. The institutional memory of the entire software portfolio lives in one queryable surface.

Deep integrations with AD Ports' ecosystem (MPay, CRM, CMS, ERP) are available through first-party agents that any IDE can reference with `@myadports-mpay`, `@myadports-crm`, and similar handles. End-to-end automation extends from BA story generation in Jira through to auto-implementation of eligible tickets, continuous PR review, and automatic fleet-wide maintenance.

> **The Core Proposition**
> **One portal. All agents. Every standard. Forever maintained.** Your BRD becomes a reviewed architecture. Your architecture becomes a running application. Your running application stays current as the organization's standards evolve. Every artifact is versioned. Every decision is traceable. Every output follows AD Ports standards by construction — and stays that way.

---

2. Vision & Positioning

The AI Portal starts from three non-negotiable premises about AD Ports' reality:

- **Developers already have IDEs they love.** GitHub Copilot, Cursor, Claude Code, JetBrains with Junie, Windsurf — the industry has invested billions in these tools and they work. A platform that asks developers to abandon them will fail on adoption.
- **AD Ports already has a mature stack.** Keycloak, AKS, PostgreSQL, GitLab, Azure DevOps, .NET, Angular — this stack works, the team knows it, production is stable on it. A platform that requires wholesale replacement of the runtime, identity layer, or data layer is solving the wrong problem.
- **The missing piece is organizational knowledge made executable and durable.** The standards documented in Confluence, the architectural patterns proven on JUL, the integration ICDs with MPay, the Helm charts validated in production — all of this lives as unstructured PDFs, tribal knowledge, and half-finished templates scattered across repositories. When a project ships, this knowledge dissipates rather than compounds. The next project re-learns it. The fourth project re-learns it differently. That gap is the real problem, and it compounds over the lifecycle, not just at bootstrap.

The AI Portal therefore takes a deliberate position: it is a **horizontal capability layer**, not a vertical platform. It exposes AD Ports' standards as MCPs, CLIs, and agent specifications that any AI coding tool can consume. It does not generate code for six platforms from a new DSL. It does not run a proprietary actor runtime. It does not replace Keycloak. It makes everything AD Ports already does faster, more consistent, more auditable — and keeps it that way as time passes.

What the Portal Is (and Is Not)

Dimension	What the AI Portal Is	What the AI Portal Is Not
Primary surface	A fabric of MCPs, agents, skills, specs that IDEs consume	A new IDE or new editor
Code generation	Template-first scaffolding consumed through existing IDEs	A new DSL that developers must learn
Runtime	Standard .NET, Angular, Postgres, AKS, Keycloak	A new proprietary runtime
Identity	Keycloak (existing AD Ports standard)	A new custom identity provider
Adoption curve	Zero — works inside Copilot/Cursor on day one	A platform requiring migration

Dimension	What the AI Portal Is	What the AI Portal Is Not
Vendor lock-in	None — outputs are plain .NET + Angular + Helm	Lock-in to a proprietary runtime or DSL
Standards enforcement	Explicit in skill/spec/instruction agents, auditable	Implicit and opaque
Legacy integration	Works with existing codebases from day one	A greenfield-only platform
Lifecycle coverage	Generation, maintenance, monitoring, migration	A one-shot scaffolding tool
Value source	Encoded AD Ports organizational knowledge	Generic industry best practice

Guiding Principles

Four disciplines anchor every design decision:

Build only what creates unique value; integrate proven open-source for everything else. The unique work is the AD Ports-specific fabric — skills, specs, instructions, ecosystem agents, the orchestration state machine, the PR review rubric calibrated to AD Ports standards. Almost everything else (LLM gateway, workflow engine, artifact storage, observability stack, container orchestrator) is integrated from best-in-class open-source.

Determinism first, LLM second. Wherever a decision can be made by a template, a rule, or a validator, it is. LLMs are used for the 20–30% of work that genuinely requires language judgment — intent extraction, ambiguity resolution, code review narrative. Deterministic templates produce 70–80% of generated code. This is explicit architectural policy because it controls cost, improves reproducibility, and makes the platform auditable.

Human-in-the-loop at every significant gate. The Portal proposes; architects and tech leads decide. Every artifact is reviewable. Every revision is versioned. Every approval is audited. The Portal never executes material changes — repository creation, infrastructure provisioning, production deployment, fleet-wide upgrades — without an explicit approval that maps to a named human role.

Every action is recorded, attributable, and replayable. The Pipeline Ledger captures the full history of every project from BRD upload through every subsequent maintenance action. Nothing the Portal does is ephemeral or unaudited. If an auditor, an incident investigator, or a new team member needs to know "what happened here and why," the answer is queryable.

---

3. The Problem We Are Solving

Every new project at AD Ports follows the same painful pattern. A business case arrives. A solution architect writes an HLD and BRD. Integration teams write ICDs with SINTECE, ASYCUDA, SICOEX, MPay, or CRM. A dev team is assigned. And then weeks disappear into bootstrap.

What "bootstrap" actually means:

- Creating the GitLab group and Azure DevOps project.
- Copy-pasting the .NET solution structure from the last project (which is already two versions behind standards).
- Re-wiring Keycloak realms, client scopes, roles, and group mappings that another team already perfected.
- Writing the same EF Core migrations or DbUp scripts, the same PostgreSQL connection resilience code, the same logging and OpenTelemetry setup.
- Scaffolding an Angular app from scratch — Nx monorepo, Native Federation, PrimeNG + Tailwind, Transloco — even though three other teams have already done it.
- Writing GitLab CI pipelines that are 80% identical to the last project but 20% different enough to break.
- Authoring Helm charts, Terraform modules, NSG rules, cert-manager configs, Kong ingress rules.
- Re-implementing notification fan-out because the existing notification service was not discovered.
- Re-implementing user management because the previous team's reusable component was not indexed.
- Re-building the same MPay integration adapter for the fifth time.
- Writing QA automation from scratch because there is no shared Playwright framework.
- Writing Postman collections by hand for deployed-system integration testing.
- Wiring SonarQube and Checkmarx into the pipeline manually, often incompletely.
- Writing Docusaurus for architecture docs, then letting it rot after go-live.

But bootstrap is only half the problem. After a project ships, the maintenance tax begins:

- **Framework upgrades** are done project-by-project, months or years apart, with every team re-learning the migration path. A .NET 6 → .NET 10 rollout across 40 AD Ports

projects means 40 separate upgrade efforts, each slightly different.

- **Security patches** from Checkmarx or Snyk findings are applied independently in every repository, with no coordination. A critical CVE in a common NuGet package triggers 40 separate remediation tickets instead of one campaign.
- **Library drift** means the same package is at different versions in different projects, producing different behavior in subtly different ways.
- **Production incidents** on shared services are discovered by each downstream team independently because there is no central health-monitoring signal.
- **Integration testing** of deployed applications is ad-hoc — one team uses hand-crafted Postman collections, another uses Insomnia, a third uses nothing and relies on manual testing in UAT.

The compounding cost is enormous. Across a portfolio of active projects, bootstrap alone represents thousands of engineer-weeks per year. Maintenance repetition likely represents even more, though it is harder to measure because it is distributed across every team rather than concentrated in obvious bootstrap sprints.

AI coding tools have not fixed this. Copilot and Cursor are brilliant at keystroke acceleration but they have no access to AD Ports' architectural standards, no view of the portfolio, and no ability to coordinate across projects. They produce generic code, and the team then spends another two weeks aligning that generic code with AD Ports standards that nobody can articulate in a prompt. And they have no concept of "upgrade all 40 projects to the next LTS."

The AI Portal closes these gaps by making AD Ports' standards a first-class asset that any AI tool can consume programmatically, **and by treating every project as a permanent, addressable entity that the platform can return to — to upgrade, to patch, to monitor, to audit — for the entire lifecycle**.

---

4. Platform Architecture Overview

The AI Portal is organized into **six logical tiers**. Each tier has a narrow responsibility and a well-defined interface to its neighbors. AI is not a tier — it runs inside every tier, from intent extraction in the Portal UI to code review in the PR agent.

The Six Tiers

Tier 1 — Experience. The web-based **AI Portal UI** where standards, MCPs, agents, skills, and policies are browsed, configured, and approved. The surface where architects upload BRD/HLD/ICDs and receive proposed architecture artifacts for review, BAs and PMs trigger

user-story generation, platform teams author and publish skills, SREs configure health monitoring, and leadership reviews fleet-wide maintenance campaigns.

Tier 2 — Capability Fabric. The shared library of **MCP servers**, **agent definitions**, **skill packages**, **specification bundles**, and **instruction sets** that encode AD Ports standards. This is the portable, consumable knowledge base — the same fabric is used by the Portal's own orchestrator and by external tools (Copilot, Cursor, Claude Code, Jira plugins, CLI integrations).

Tier 3 — Orchestration. The **Orchestration Agent** — the single entry point that reads intent, consults the capability fabric, produces an architecture proposal, and after approval delegates to specialized sub-agents. Backed by a deterministic workflow engine for state-machine flow, multi-agent collaboration patterns for team coordination, and provider-agnostic LLM routing.

Tier 4 — Specialist Agents. The domain-specific agents — Frontend, Backend, Database, Integration, DevOps, Infrastructure, Security, Observability, QA Automation, Deployed-System Verification, Migration, Framework Migration, Fleet Upgrade — plus ecosystem-specific agents (MPay, CRM, CMS, ERP, Keycloak, MDM, Notifications) — each owning one slice of the delivery or maintenance pipeline.

Tier 5 — Execution & Governance. The layer that actually creates repositories, runs pipelines, applies infrastructure, executes fleet upgrades, and enforces guardrails. Hooks fire on every significant action to enforce role-based access, budget limits, and safety rules. All artifacts are versioned and snapshotted for stage-by-stage recall. The PR Review Agent, the Vulnerability Radar, the Framework-Obsolescence Monitor, the Service Health Monitor, and the Pipeline Ledger all live here as continuous governance processes.

Tier 6 — Shared Services. Production services that new applications inherit for free — **Keycloak** (identity), **MDM** (master data), **Notification Hub** (email/SMS/push), **Audit Service**, **File Service**, **Common QA Framework**. New projects reference these rather than rebuilding them. The Portal itself also consumes these services for its own operation, dogfooding the standards.

The Intent-to-Running-Application Pipeline

Every project flows through a nine-stage pipeline. Every stage is recorded in the Pipeline Ledger (Section 13) with full artifact linkage and approver identity.

(1) Intent capture — architect uploads BRD/HLD/ICD, or references them from an IDE via `@adports-orchestrator analyze`. **(2) Understanding** — the Orchestration Agent extracts business goals, bounded contexts, integration requirements, non-functional requirements, and compliance constraints. **(3) Standards consultation** — Skill, Spec, and Instruction agents return relevant AD Ports patterns. **(4) Proposal generation** — architecture diagrams (draw.io), component decomposition, integration map, infrastructure plan, QA automation

plan, security pipeline plan, and a Docusaurus spec site are drafted. **(5) Review cycle** — the technical lead reviews, comments, and approves or requests modifications. Every revision is versioned; every approval is signed and timestamped. **(6) Delegation** — the Orchestration Agent breaks the approved plan into work packages for specialist agents. **(7) Parallel execution** — specialist agents create repos, scaffold code, write secure pipelines, provision infrastructure, wire shared services, generate unit and integration tests, generate Playwright automation suites, and generate Postman-style integration test collections. **(8) Deployed-system verification** — the fresh deployment is tested against the generated integration collection across every configured environment (dev, QA, staging) with sample payloads to prove end-to-end working. **(9) Handover & Registration** — the team receives repo URLs, running staging URLs, architecture docs, test reports, QA automation reports, security baseline reports, and a prioritized backlog of next-step stories. The project is registered in the **Project Registry** (Section 26) and enters ongoing maintenance coverage — framework migration, vulnerability radar, fleet upgrade, health monitoring all activate automatically.

Component Catalog (High Level)

Component	Tier	Role
Portal UI	Experience	Web UI for browsing capabilities, uploading docs, approving architectures, monitoring health
MCP Registry	Capability	Catalog of internal MCP servers — discoverable by Copilot, Cursor, Claude Code
CLI Distribution	Capability	`adports-ai` CLI installable via npm/winget for terminal and IDE use
Skill / Spec / Instruction Agents	Capability	Packaged AD Ports knowledge, contracts, and policies
Orchestration Agent	Orchestration	Master agent — reads intent, plans, delegates, consolidates
Backend / Frontend / DB / Integration Agents	Specialist	Scaffold CQRS services, MFEs, data stores, messaging
DevOps / Infra / Security / Observability Agents	Specialist	Pipelines, cloud resources, security gates, telemetry

Component	Tier	Role
QA Automation Agent	Specialist	Playwright / API contract / load / accessibility test generation
Integration Test Agent	Specialist	Postman-style collections, multi-environment verification of deployed systems
BA/PM Story Agent	Specialist	Jira / Azure DevOps story generation from BRDs
Ticket Implementation Agent	Specialist	Picks up a Jira ticket, implements it, commits, marks done
Migration Agent / Framework Migration Agent	Specialist	Legacy modernization, framework upgrades
Fleet Upgrade Agent	Specialist	Coordinated upgrades across every project in the portfolio
Service Health Monitor	Governance	Live health checks of every AD Ports service, notifications on degradation
MPay / CRM / CMS / ERP Agents	Specialist	Ecosystem adapters — plug-and-play integration
PR Review Agent	Governance	Scores pull requests, auto-rejects below threshold
Vulnerability Radar / Framework Obsolescence Monitor	Governance	Continuous portfolio scanning
Project Registry	Governance	Authoritative inventory of every generated project and its stack
Pipeline Ledger	Governance	Immutable record of every stage, artifact, approval
Hook Engine	Governance	Role-based guardrails, budget limits, policy enforcement
Artifact Store (Shared Project Context)	Governance	Versioned artifacts, team-shared context for in-flight projects

Component	Tier	Role
Keycloak / MDM / Notification Hub / Audit / File / QA Framework	Shared Services	Out-of-box capabilities new projects consume immediately

---

5. The AI Portal — Central Control Plane

The AI Portal is the web UI and API surface where everything else is managed. It is **not where code is written**. Code is still written in the developer's IDE. The Portal is where **capabilities are published, governed, and composed**.

What the Portal Shows

The left navigation is organized into categories that mirror the capability fabric.

Agents. Every agent is listed with its description, input/output schema, required permissions, LLM routing preferences, budget caps, tools it consumes, and version history.

MCP Servers. Every internal MCP server — Keycloak, GitLab, AKS, MPay, Jira, Azure DevOps, SonarQube, Checkmarx, Postman/Newman, etc. — is listed with its connection URL, tool catalog, authentication method, and which roles can consume it. A "Connect from your IDE" button generates a one-click config snippet.

APIs & CLIs. The `adports-ai` CLI is documented with install instructions. Every sub-command is listed with examples.

Skills / Specifications / Instructions. Each versioned, reviewed, and mapped to technology stacks.

Architectural Patterns. Higher-level compositions.

Hooks & Guardrails. Every rule the Hook Engine evaluates, plus the audit log of firings.

Projects. The running catalog, showing for each project: its current stage, last artifact version, the review status, the linked Jira/Azure DevOps board, QA coverage, security posture, framework currency status, health status of the deployed services, and drill-down links to every stored artifact and every pipeline ledger entry.

Project Registry. The authoritative inventory of every generated project with its full stack fingerprint (Section 26).

Fleet Maintenance. The surface for launching and monitoring fleet-wide upgrade campaigns (Section 20).

Service Health. Live health dashboard for every AD Ports service under monitoring (Section 24).

Pipeline Ledger Explorer. Queryable audit surface — "show me every stage 5 approval for projects in the JUL cluster in the last quarter," "show me every artifact linked to Jira epic DGD-001," "show me every approver who signed off on production Keycloak changes."

Audit & Observability. OpenTelemetry traces, agent cost dashboards, agent success rates, PR review scores across the portfolio, guardrail firing frequency.

Who Uses the Portal

The Portal is multi-persona. Each persona sees a tailored view:

- **Solution Architect.** Onboards new projects, reviews architecture proposals, retrieves past artifacts, initiates legacy migrations.
- **Technical Lead.** Reviews and approves stage artifacts, opts project into auto-implementation, configures PR review thresholds, monitors project health.
- **Platform Engineer / Principal Architect.** Publishes and updates skills, specs, instructions; registers new MCP servers; tunes agent routing and budgets; launches fleet upgrade campaigns.
- **Business Analyst / Project Manager.** Decomposes BRDs into user stories, inspects story quality, publishes to Jira/Azure DevOps.
- **Developer.** Consumes the Portal primarily through their IDE via MCPs; uses the Portal UI to retrieve context, inspect PR review feedback, and track auto-implemented tickets.
- **QA Engineer.** Reviews and extends generated test suites, inspects verification reports across environments, triages flakes.
- **Security Engineer.** Monitors vulnerability posture across the portfolio, configures security baselines, reviews security-gated PRs.
- **SRE / Operations.** Configures health monitoring, receives and responds to service-down notifications, investigates incidents with support from the incident post-mortem assistant.
- **Compliance Officer / Internal Auditor.** Queries the Pipeline Ledger for evidence, exports audit reports, reviews approval trails.
- **Engineering Leadership.** Receives weekly digests, monitors portfolio health trends, approves fleet maintenance campaigns.

Every persona's visibility is governed by OpenFGA-backed permissions tied to Keycloak group membership.

---

6. The Capability Fabric — MCPs, APIs, CLIs, Skills, Specs, Instructions

The capability fabric is the content of the Portal — the reusable assets that make AI orchestration useful.

MCP Servers — Programmatic Access from Any AI Tool

MCP (Model Context Protocol) is the emerging industry standard for exposing tools to LLM-based agents. The AI Portal ships a suite of internal MCP servers. The same MCP servers are consumed by the Portal's own Orchestration Agent **and** by external tools — Copilot, Cursor, Claude Code, or any other MCP-aware client — simply by adding the server URL to the client's configuration.

Examples of MCP servers in the fabric:

- **Keycloak MCP** — list realms, create client, add role, configure group, export realm JSON.
- **GitLab / Azure DevOps MCP** — create project, add variables, protect branches, run pipeline, read logs.
- **AKS MCP** — list namespaces, apply Helm chart, read pod logs, describe pod.
- **Postgres MCP** — create database, apply migration, run schema diff, generate seed script.
- **Jira MCP** — create epic, create story, transition ticket, link commits, update status.
- **Azure Boards MCP** — same surface for teams on Azure DevOps work items.
- **SonarQube MCP** — fetch quality gate, run analysis, retrieve issues, track technical debt.
- **Checkmarx MCP** — trigger SAST scan, fetch findings, manage scan configurations.
- **Snyk / Trivy MCP** — SCA and container scanning surface.
- **MPay MCP** — get integration schema, test sandbox auth, fetch OAuth client credentials, validate webhook.
- **CRM / CMS / ERP MCPs** — one per shared business system.
- **AD Ports Standards MCP** — fetch coding standard, list approved libraries, retrieve naming conventions.
- **Docusaurus MCP** — scaffold a docs site, publish an architecture page, version a release.

- **Draw.io MCP** — generate a diagram from a component list, fetch stencils, export PNG/SVG.
- **Playwright MCP** — generate test suites from scenarios, run against staging, retrieve reports.
- **Postman/Newman MCP** — generate, run, and validate Postman collections across environments.
- **Health Check MCP** — register, query, and manage service health probes.
- **Project Registry MCP** — query and update the fleet inventory.
- **Pipeline Ledger MCP** — query the immutable audit surface.
- **Vault MCP** — fetch a secret, rotate a credential, list access grants.

The value of MCP as the integration primitive is twofold. First, a developer working in Cursor with `@adports-keycloak create a realm for the new billing service` gets the same result as the Orchestration Agent running unattended in the Portal. Second, MCP is an industry standard — Anthropic, Google, and the broader agent community all back it — so adopting it means we inherit every future MCP-aware tool for free.

REST / gRPC APIs

Not every consumer is an LLM-based agent. The Portal exposes traditional REST and gRPC APIs for CI/CD pipelines, dashboard applications, back-office tools, and webhook consumers. Every API endpoint wraps the same underlying capability as the equivalent MCP tool.

CLI — `adports-ai`

```bash
adports-ai analyze ./docs/BRD.pdf ./docs/HLD.pdf ./docs/ICD-mpay.pdf
adports-ai plan --pattern=angular-mfe-dotnet-cqrs --target=aks
adports-ai execute --approve
adports-ai ticket implement JUL-1234
adports-ai qa generate --scope=declaration-service
adports-ai verify-deployed --env=staging --project=dgd
adports-ai fleet upgrade --from=dotnet6 --to=dotnet10 --dry-run
adports-ai health watch --cluster=jul
adports-ai ledger query --project=dgd --stage=5
```

The same CLI is what the Portal UI invokes under the hood, and what Copilot or Cursor invoke when they want to delegate heavy work.

Skills, Specifications, Instructions

Skills are packaged how-tos (keycloak-realm-setup, dotnet-cqrs-scaffold, angular-nx-microfrontend, playwright-e2e-baseline, postman-integration-baseline, fleet-upgrade-playbook-dotnet, health-check-baseline). **Specifications** are machine-readable contracts (adports-keycloak-realm.schema.json, adports-helm-chart.values.schema.json, mpay-integration.openapi.yaml). **Instructions** are policy text (coding-standards-csharp.md, security-baseline.md, framework-lifecycle-policy.md, fleet-campaign-policy.md, health-monitoring-policy.md). All versioned, reviewed, reviewed on a cadence.

---

7. IDE & Agent Integration (Copilot, Cursor, Claude Code, Terminal)

A core design principle: **developers keep their IDEs**. The AI Portal meets them where they are.

The Connection Pattern

Developers add two things to their IDE configuration:

- **AI Portal MCP server URLs.** Paste relevant ones into the MCP client config.
- **The `adports-ai` CLI.** Installed once, available from terminal and from any IDE's shell integration.

Shortcut Handles

- `@adports-orchestrator` — analyze, plan, execute, review.
- `@adports-backend`, `@adports-frontend`, `@adports-devops`, `@adports-qa`, `@adports-security`, `@adports-keycloak` — specialist agents.
- `@adports-verify` — run or extend the deployed-system integration test collection.
- `@adports-fleet` — inspect fleet upgrade campaigns, opt a project in or out.
- `@adports-health` — query live service health.
- `@myadports-mpay`, `@myadports-crm`, `@myadports-cms`, `@myadports-erp` — ecosystem integrations.
- `@adports-jira` — create or transition tickets.
- `@adports-review` — pre-flight code review before pushing.
- `@adports-ledger` — query the pipeline ledger.

The Two-Way Relationship

The Portal does not compete with Copilot — it composes it. When the Orchestration Agent is executing a large plan, it may delegate to a general-purpose coding agent via MCP, then collect the result back.

---

8. The Orchestration Agent & Agent Hierarchy

The Orchestration Agent is the single logical entry point for complex multi-step work. It is backed by a deterministic workflow engine rather than a free-form LLM loop.

The Four Tiers

Tier 1 — Orchestration. One central orchestrator per project. Owns the plan, the approvals, the delegation, and the consolidation of results.

Tier 2 — Domain Specialists. Backend, Frontend, Database, Integration, DevOps, Infrastructure, Security, Observability, QA Automation, Integration Test, Migration, Framework Migration, Fleet Upgrade.

Tier 3 — Ecosystem Specialists. MPay, CRM, CMS, ERP, JUL event bus, PCS, Keycloak, MDM, Notifications.

Tier 4 — Micro-Agents and Tools. Leaf-level workers — lint sub-agent, schema-diff sub-agent, draw.io generator, Postman collection generator, commit-message formatter.

How a Request Flows

The Orchestration Agent receives a request and performs deterministic phases: intent extraction, standards retrieval, pattern selection, component decomposition, artifact synthesis, human review gate, delegation and consolidation. At every phase, hooks fire, the Pipeline Ledger records the event, audit logs are written, and OpenTelemetry traces propagate through every sub-agent invocation.

LLMs are used within each node of the state machine for the 20–30% of steps that genuinely need language judgment. The rest is code.

---

9. Specialized Delivery Agents

Each specialist agent is a focused, tool-rich worker. It receives a scoped task from the orchestrator, consults its skills and specs, produces outputs, validates them, and reports back.

Backend Agent produces a production-ready .NET solution following AD Ports CQRS conventions — Domain, Application, Infrastructure, Api, Tests projects; MediatR handlers; FluentValidation; EF Core; OpenTelemetry; health checks; Dockerfile; passing `dotnet build` + `dotnet test` on first run.

Frontend Agent produces an Angular or React micro-frontend — Nx workspace with Native Federation, PrimeNG/Material + Tailwind, Transloco, Keycloak auth, Jest/Vitest unit tests, Playwright E2E skeletons.

Database Agent provisions Postgres/SQL/MongoDB with DbUp or EF Core migrations, seed scripts, schema-diff comparison, environment-specific connection-string templates.

Integration Agent wires MassTransit + RabbitMQ, inbox/outbox patterns, Camunda BPMN, Saga definitions, dead-letter handling, idempotency keys.

DevOps Agent produces GitLab/Azure DevOps pipelines with SonarQube + Checkmarx + Snyk + Trivy baked in, variables wired to Vault, approval gates for production, templates imported from central repositories so security fixes propagate automatically.

Infrastructure Agent writes Pulumi or Terraform for Azure resources — AKS namespace, Postgres Flexible Server, Key Vault, NSG rules, Kong ingress routes, cert-manager, observability wiring.

Observability Agent adds OTEL instrumentation, Grafana dashboards for latency/error/throughput/business KPIs, alert rules for SLO breaches, log correlation IDs through Kong/services/RabbitMQ.

Specialist agents run **in parallel wherever dependencies allow**. The orchestrator maintains the dependency graph and streams progress to the Portal UI and the Pipeline Ledger in real time.

---

10. Worked Example — "Generate a New JUL-Style Microservice Application"

Business context. ARCCLA requests a new "Dangerous Goods Declaration" (DGD) module. Cargo operators submit DGD forms; the system classifies them, integrates with SINTECE, and charges fees via MPay. Tech lead has BRD, HLD sketch, and ICD.

Step 1 — Intent Capture. Architect opens the Portal, clicks **New Project**, uploads documents, picks pattern **Angular MFE + .NET CQRS + Camunda + RabbitMQ + Postgres**, submits. Intent-capture event written to Pipeline Ledger.

Step 2 — Orchestrator Analysis. Structured intent extracted: bounded contexts, integrations (SINTECE, MPay), non-functional requirements, users. Skill, spec, and instruction agents queried; context bundle assembled. Each retrieval recorded in ledger with artifact hashes.

Step 3 — Architecture Proposal. Orchestrator synthesizes draw.io component diagram, integration sequence diagram, infrastructure plan, QA automation plan, secure-pipeline plan, initial Jira epic/story decomposition, and a Docusaurus spec site preview. Everything versioned; each artifact gets a content hash recorded in the ledger.

Review cycle. Tech lead comments: "Use shared `jul-reference-data` service for HS code lookup." Orchestrator stores v0.1, applies feedback, publishes v0.2. Tech lead approves. The approval is digitally signed and written to the ledger with the approver's Keycloak identity, timestamp, and the exact artifact hashes being approved.

Step 4 — Parallel Execution. Backend, Frontend, Database, Integration, DevOps, Infrastructure, QA Automation, Integration Test, Security, Observability, BA/PM Story agents fire in dependency-aware parallel.

Each specialist agent's output is a ledger entry with the commit SHA, artifact hash, and validation results. The Backend Agent produces three .NET solutions; the Frontend Agent adds a new `dgd-mfe` remote to the existing JUL Nx workspace; the Database Agent adds migrations; the Integration Agent wires Camunda and event contracts; the DevOps Agent creates pipelines with full security scanning; the Infrastructure Agent provisions resources via Pulumi; the QA Automation Agent generates Playwright suites, k6 load tests, and Axe accessibility scans; the **Integration Test Agent** generates a Postman collection covering every endpoint with sample request payloads for three environments (dev, QA, staging); the Security Agent configures Checkmarx presets and SonarQube quality gates; the Observability Agent publishes dashboards; the BA/PM Story Agent publishes the backlog to Jira.

Step 5 — Deployed-System Verification. Once the scaffold is deployed to the dev and QA environments, the Integration Test Agent runs the generated Postman collection against each environment via Newman and produces a cross-environment report: which endpoints passed, which failed, response-time percentiles, any schema drift. The report is linked to the project in the Portal.

Step 6 — Handover & Registration. Orchestrator publishes the handover bundle (repo URLs, staging URLs, Docusaurus site, test reports, Postman collection URLs, Jira board URL, runbook, next-steps markdown). The project is registered in the **Project Registry** with its full stack fingerprint (.NET 9, Angular 20, Postgres 16, Helm chart versions, NuGet and npm versions). From this moment on, the project is covered by the Fleet Upgrade Agent, Vulnerability Radar, Framework Obsolescence Monitor, Service Health Monitor, and PR Review Agent automatically.

The entire journey is permanently recorded in the Pipeline Ledger, queryable via the UI, the CLI, or the Ledger MCP.

Letting agents create real repositories, real pipelines, and real Azure infrastructure would be irresponsible without guardrails. The Hook Engine is where every sensitive action is gated.

Hook Model

Pre-hooks fire before an action and can block or require approval. Post-hooks fire after the action and can alert, audit, or trigger remediation. Budget hooks enforce LLM cost, API call, or compute limits. Hooks are written as declarative policy (OPA/Rego) and live in the Portal alongside instructions.

Examples of Shipped Hooks

- **Role-based provisioning.** Only `architect` or `principal-engineer` Keycloak groups can trigger Pulumi apply for production subscriptions.
- **Project budget.** Every project has an LLM spend budget per orchestration cycle.
- **Forbidden operations.** No agent can delete production databases. Ever.
- **Two-person approval.** Any change to the shared Keycloak realm for production requires two architect sign-offs.
- **Sensitive data redaction.** Before sending BRD/HLD content to an external LLM, redaction hooks strip UAE-PASS national IDs, MPay merchant secrets, classified references. Classified projects route to an on-premises LLM that does not egress data.
- **Compliance scope.** NESA-scoped projects automatically pull in NESA-specific instructions and validation steps.
- **Auto-implementation bounds.** The Ticket Implementation Agent (Section 22) can only implement tickets with estimate below a threshold and only in opted-in projects.
- **Fleet campaign bounds.** Fleet upgrades above a declared risk tier require CAB (Change Advisory Board) approval.
- **Health-monitor action bounds.** Auto-remediation (restart pod, clear cache) requires opt-in per service; notification-only is the default.

Audit

Every hook evaluation is logged with the decision, the actor, the timestamp, and the context that was evaluated. Fed into the Pipeline Ledger.

---

12. Artifact Generation, Versioning & Shared Project Context

Every stage of every orchestration produces artifacts. Every artifact is versioned. The team must be able to return to any stage, inspect what was there, compare to a later stage, and fork a new branch from any point — and every team member working on the project must see the same context.

Shared Project Context — One Team, One Memory

Today, when multiple developers collaborate on a project with AI tools, each developer has their own conversation history. Context is not shared. Agent decisions made in one developer's session are invisible to another. The result: fragmented AI collaboration and inconsistent outputs.

The Portal fixes this with **shared project context** — a Redis-backed context layer scoped to the project, visible to every team member with project access. When Developer A asks the Orchestration Agent to evaluate a design choice, the resulting context, artifacts, and decisions are immediately visible to Developer B. When Developer C references `@adports-orchestrator what did we decide about retry semantics?`, the answer comes from the same shared context, not from a fresh session.

How shared context works:

- **Scope.** Context is keyed by project. All team members with project role visibility share the same active orchestrator context.
- **Sessions.** An orchestrator session is a sequence of prompts, responses, artifact references, and decisions. Sessions are persisted to Redis hot, archived to cold storage after a configurable retention window.
- **Participation.** Every prompt includes the identity of the team member who issued it. Every response, artifact, or decision is attributed. The team sees the full conversation thread.
- **Cross-project referencing.** A developer working on Project A can explicitly reference context from Project B (with appropriate permissions): `@adports-orchestrator reuse the MPay retry pattern we agreed on in project jul-dgd`. The orchestrator fetches the relevant session excerpt from Project B's context with provenance attribution.
- **Privacy and compartmentalization.** Access to shared context is governed by OpenFGA — a member of Project A does not automatically see Project B's context. Cross-project access requires explicit permission grant.
- **Scrubbing.** Sensitive values (credentials, secrets, PII) are redacted from shared context at capture time. Hooks enforce this.

Why this matters: It transforms AI collaboration from individual tool use into a team capability. When a new team member joins, they inherit the accumulated context of prior conversations. When a decision was made weeks ago, the rationale is still there. When three developers work in parallel, they do not produce three divergent AI-generated answers.

What Counts as an Artifact

- The structured intent extracted from BRD/HLD/ICD.
- The context bundle pulled from skills, specs, instructions.
- Each proposal version (architecture doc, draw.io diagrams, OpenAPI drafts, Docusaurus snapshots, QA plan, security plan).
- Each human review comment set.
- Each specialist-agent output — generated code as a Git commit SHA plus full artifact tarball, Pulumi preview, pipeline YAML, Playwright project, Postman collection, Newman reports per environment.
- The final handover bundle.
- Every downstream maintenance action — fleet upgrade campaign artifacts, framework migration branches, ticket implementations, PR reviews, vulnerability radar findings.

Storage Design

Hot cache (Redis cluster). Active project context, current session state, recent artifact versions. Keyed as `project:{id}:session:{s}:turn:{t}` and `project:{id}:stage:{stage}:version:{v}`. Sub-millisecond recall for interactive use.

Cold storage (Azure Blob). Every artifact version, every orchestrator session archive, indefinite retention. Immutable bucket with legal-hold where compliance requires it.

Metadata database (Postgres). Relational catalog — project → stage → version → artifact pointer → approver → timestamp → hash. Query surface for the Portal UI's timeline view.

Event log (Kafka). Append-only stream of stage transitions and context events. Drives the real-time timeline view, enables replay.

Context Recall in Practice

A developer, months after a project shipped, wonders why the team chose Camunda over a Saga. They open the project in the Portal, jump to **Stage 3 — Pattern Selection, version 0.2**, and read the orchestrator's rationale, the alternatives it considered, and the review comment from the architect that locked the decision in.

A project is audited by internal compliance. The auditor retrieves the full artifact trail and has a complete evidence package without the engineering team producing a single new document.

---

13. The Immutable Pipeline Ledger — Recording Every Stage, Artifact, and Approval

The Pipeline Ledger is the most important governance surface in the Portal. It is the authoritative, immutable, cryptographically-chained record of everything the Portal has ever done for every project. Every proposal, every suggestion, every approval, every artifact, every delegation, every deployment is written to the ledger. Nothing is ever removed.

What Gets Recorded

For every stage of every project, the ledger captures:

- **Stage identity.** Project ID, stage number, stage name, timestamp, orchestrator version.
- **Inputs.** Hash of every input document (BRD, HLD, ICD), the retrieved context bundle (skills, specs, instructions consulted with their version IDs), any referenced cross-project context with attribution.
- **Agent actions.** Every LLM call with provider, model, token counts, cost; every tool invocation with parameters and result hashes; every hook evaluation with decision and rationale.
- **Artifacts produced.** Hash and cold-storage pointer for every artifact generated at this stage, linked to the preceding and succeeding stages.
- **Approvals.** Approver Keycloak identity, role, timestamp, the exact artifact hashes being approved, any comments. Digitally signed per approver.
- **Change requests.** Requester identity, requested change text, the artifact hashes being modified, the resulting diff, the downstream revision IDs.
- **Delegations.** Which specialist agents were invoked, with what work packages, against what dependencies, for what budget.
- **Execution results.** Every specialist agent's success/failure outcome, error messages, retry counts, consolidated output.

- **Deployments.** Environment, Kubernetes namespace, Helm release name, image digest, timestamp, deployer identity.
- **Maintenance actions.** Every subsequent fleet upgrade, framework migration, security patch, ticket implementation, PR review that touches the project.

How the Ledger Is Stored

Append-only event stream. Every ledger event is written to an EventStoreDB-backed stream for the project plus a portfolio-wide stream. No event is ever updated or deleted.

Cryptographic chaining. Each ledger entry includes a hash of the previous entry for the same project, producing a tamper-evident chain. Periodic checkpoint hashes are notarized (for example, to an internal blockchain ledger or a trusted timestamping service) so any tampering would be detectable.

Digital signatures for approvals. Approvals are signed using the approver's Keycloak-issued credential (X.509 or similar). Signature validation is part of the ledger-query tooling.

Indexed query surface. A secondary Postgres index is built from the event stream for fast queries — by project, stage, approver, artifact hash, date range, Jira ticket reference, framework campaign, service affected. The Pipeline Ledger Explorer in the UI sits on top of this index.

What This Enables

Compliance and audit. An internal auditor can answer "show me every stage-5 approval for JUL projects in the last quarter, with approver identity and the exact artifact hashes approved" in seconds. An external auditor under ISO-27001 or NESA review gets a full evidence package from a single export.

Post-incident analysis. When an incident occurs, the on-call engineer can query the ledger for "what changed on service X in the last 48 hours" and get every deployment, every pipeline run, every Helm chart update, every framework patch — with the originating agent action and approver.

Traceability from BRD to production. Every line of production code can be traced backward through the ledger to the commit → the PR → the Jira ticket → the story → the epic → the architectural stage → the BRD paragraph that motivated it. And forward — from any BRD paragraph to the code and deployment that implemented it.

Change lineage. Why was this Helm chart version pinned? Query the ledger: stage-7 delegation to Infrastructure Agent on date X, with a rationale linked to CVE-Y raised by Vulnerability Radar, approved by architect Z with comment "pinning until base image upgrade in Q3."

Replayability. If an agent release has a regression, any previous project stage can be replayed with a different agent version to validate the fix — inputs are preserved in the ledger, so reproducibility is deterministic.

Organizational memory. Institutional knowledge of architecture decisions, trade-offs, and rationale no longer decays over time. Every decision is queryable forever.

Recommended Additions to Strengthen the Ledger

Beyond the core captures above, the ledger is designed to include:

- **Risk-tier annotation.** Each stage is tagged with a risk tier (low/medium/high/critical) derived from the Hook Engine's evaluation. Portfolio dashboards show risk distribution over time.
- **Compliance tag propagation.** A project declared NESA-in-scope tags every downstream stage, making compliance audits a single filtered query.
- **Cross-project reference graph.** When Project B's orchestrator consumes context from Project A, a directed edge is recorded. Over time this produces a graph of knowledge reuse across the portfolio — useful for identifying high-value patterns worth promoting to skills.
- **Fleet-campaign membership.** When a project participates in a fleet upgrade campaign, every ledger entry in that campaign carries the campaign ID, producing a cross-project view of the campaign.
- **Emergency-override trail.** Any override of a Hook Engine rejection requires an explicit justification recorded in the ledger with the overrider's identity, reason, and a follow-up ticket for review. Overrides are surfaced in the weekly leadership digest.
- **Agent version provenance.** Every agent invocation records the exact agent version and the exact skills/specs/instructions versions consulted. A regression traced to a specific agent release can be rolled back fleet-wide.
- **Sign-off chains for regulated workflows.** Some AD Ports workflows (ARCCLA regulatory touchpoints, UAE customs) require multi-party sign-off. The ledger enforces the declared chain and refuses to advance until every party has signed.

---

14. Review & Approval Workflow

Humans stay in the loop. The AI Portal proposes; architects and tech leads decide. Every approval is a first-class Pipeline Ledger event.

The Review Surface

Every stage that produces artifacts emits a review task. The reviewer sees:

- A **side-by-side diff** against the previous version.
- A **rationale pane** where the orchestrator explains its significant choices with links to consulted skills, specs, instructions.
- A **cost and time estimate** for downstream execution.
- A **comment thread** scoped to the stage.
- **Linked artifacts** — every artifact version with its hash, its predecessors, its consumers.
- Buttons: **Approve**, **Request changes** (with comments), **Reject and stop**.

The Revision Loop

A "request changes" review sends comments back to the orchestrator, which generates a new version preserving everything else. The reviewer sees a diff of just the requested changes plus any downstream effects. Every revision is a ledger event. Approval may happen on version 0.2 or 0.5 — the timeline captures every step.

Escalation and Two-Person Review

For changes that breach the Hook Engine's two-person rule (production Keycloak, production infra, shared services, high-risk fleet campaigns), the review requires two approvers. The UI shows this; the orchestrator will not proceed until both sign-offs are in the ledger.

Read-Only Observers

Security, compliance, and product stakeholders can be added as read-only observers. They see the timeline and all artifacts, cannot approve or change. This is often how security reviews happen in practice.

---

15. Out-of-the-Box Shared Services (Identity, MDM, Notifications, QA Framework)

New projects do not implement identity, master data, notifications, or QA framework from scratch. The Portal ensures they consume AD Ports' shared services instead.

Identity — Keycloak

Every new project is wired to Keycloak at scaffold time. Backend and Frontend agents add

required client configurations, roles, groups, audience claims following AD Ports conventions.

Master Data Management — MDM Service

Reference data (country codes, HS codes, port codes, organization master) lives in the central MDM service. Backend Agent wires each new service to the MDM client library.

Notification Hub

Email, SMS, and push notifications go through the central Notification Hub. Backend Agent wires the `AdPorts.Notification.Client` NuGet package.

Audit Service

Every service emits audit events to the central Audit Service via a standard schema.

File Service

Uploads, downloads, and secure file sharing go through the central File Service with virus scanning, tenant isolation, expiring URLs.

Common QA Framework

A shared Playwright harness, API contract test harness, load-test baseline, and Postman collection template are provided as NuGet + npm packages. New projects inherit the AD Ports Page Object Model, Keycloak authentication helpers, data seeding helpers, the standard reporting format.

The Net Effect

A new microservice built through the Portal inherits identity, reference data, notifications, audit, file handling, and QA automation as scaffolded dependencies, not as features to build. These six services alone account for weeks of work per project when reimplemented by hand.

---

16. QA Automation Agent — Playwright, Load, Accessibility & Visual Regression

The QA Automation Agent is a first-class specialist, not an afterthought. Every new project gets a working automation suite from day one.

What the Agent Produces

End-to-end tests (Playwright). A Playwright project using the AD Ports common QA framework, pre-wired with Keycloak auth, data seeders, Page Object Model. The agent

reads BRD user journeys, HLD component structure, and OpenAPI contracts to generate:

- One Page Object per MFE screen.
- Test fixtures per user role.
- Happy-path E2E scenarios from BRD acceptance criteria.
- Negative-path scenarios for common failure modes.

API contract tests. From OpenAPI contracts — every endpoint gets a contract test validating schema, status codes, and error shapes.

Load tests (k6 or Gatling). Calibrated to non-functional requirements from the BRD.

Integration tests (xUnit / NUnit). Per service, against Testcontainers-backed Postgres and RabbitMQ, covering CQRS handlers, integration-event publishing, inbox/outbox idempotency.

Accessibility tests. Axe-core inside Playwright scans every screen for WCAG violations.

Visual regression tests. Optional per project via Percy, Chromatic, or Playwright snapshot comparison.

Scenario-Driven, Not Implementation-Driven

The agent reads BRD user journeys, acceptance criteria on Jira stories, OpenAPI contracts, and MFE route manifests. From these it synthesizes test code reflecting business intent. When the BRD says "operator exceeding quota cannot submit a DGD," the agent produces `operator_over_quota_cannot_submit_dgd` that seeds the quota state, attempts submission, and asserts the correct error surface.

Integration with the Pipeline

Every pipeline generated by the DevOps Agent includes test stages: unit tests → API contract tests → integration tests against Testcontainers → deployment to preview → Playwright E2E against preview → accessibility scan → optional load test. Pipeline fails at any stage that regresses below the project's declared quality bar.

Post-Upgrade Verification

When the Fleet Upgrade Agent (Section 20) applies a campaign to a project, the full generated test suite runs against the upgraded branch. A campaign does not merge unless the upgraded code passes at least the same test bar as the baseline.

---

17. Deployed-System Verification — Postman-Style Integration Testing

Unit tests verify code in isolation. E2E tests verify the application from the user's

perspective in a browser. But a third class of verification is needed: **is the deployed API actually working across every environment?** This is where most hand-written Postman collections historically live at AD Ports, produced ad-hoc and rarely maintained.

The **Integration Test Agent** treats this as a first-class automated capability.

What the Agent Produces

For every new project, the agent generates:

A complete Postman collection with one request per API endpoint, organized by service and by resource. Each request includes:

- Full URL with environment-variable placeholders.
- Authentication headers wired to Keycloak (with token acquisition as a pre-request script).
- Sample request payloads derived from the OpenAPI schema and from BRD examples — not just schema-valid garbage, but realistic data that reflects actual business scenarios (a DGD for "dangerous chemicals from Luanda to Lobito," a payment settlement for "USD 2500 operator margin").
- Test scripts asserting status codes, response schema, business invariants, and performance SLOs.
- Negative scenarios — bad auth, malformed input, forbidden operations — with expected error responses.

Environment files per deployment target — dev, QA, staging, UAT, production — with environment-specific URLs, client IDs, and test-data references. Secrets are never committed to the environment files; they are pulled from Vault at run time.

Data fixtures and seeders. For tests that require specific test data (an existing DGD in `Triaged` state to test transitions), the agent generates seed scripts that populate the environment before the collection runs and tears down after.

A Newman-driven pipeline stage. The pipeline includes a post-deployment stage that runs the Postman collection against every configured environment in sequence — dev first, QA next, staging last — and fails the deployment if any environment regresses.

Multi-Environment Coverage

Every generated collection is executable against every environment from day one. The agent produces a matrix report:

Endpoint	Dev	QA	Staging	UAT	Production
`POST /declarations`	✓	✓	✓	N/A	N/A
`GET /declarations/{id}`	✓	✓	✓	N/A	N/A
`POST /declarations/{id}/classify`	✓	✓	✗	N/A	N/A
…					

The matrix is published to the Portal and refreshed on every deployment. Environment-specific failures (a contract that works in dev but fails in staging) are immediately visible.

Synthetic Data and PII Safety

The agent uses **synthetic data** by default — data that is realistic in shape but compliance-safe. Real customer data is never used in test collections. The synthetic-data service (Section 29) generates schema-accurate fixtures on demand.

Self-Updating Collections

When a new endpoint is added to a service, the Integration Test Agent regenerates the collection incrementally — it appends the new endpoint with generated sample payloads and preserves manual customizations the team may have added. When a contract changes, the agent flags the divergence on the PR and proposes updated test scripts.

Skills That Back the Agent

- **postman-newman-adports-baseline** — collection structure, environment file schema, pre-request scripts, Keycloak auth helpers.
- **sample-payload-generation** — how to produce realistic business-domain payloads from OpenAPI schemas plus BRD examples.
- **multi-environment-matrix-reporting** — how to aggregate Newman results across environments and surface them in the Portal.

Alternative Tooling

For teams that prefer other tools, the agent can emit equivalent artifacts for Bruno, Insomnia, Hoppscotch, or REST Client. Postman + Newman is the AD Ports default because it is already widely used.

---

18. Security Framework Integration — SonarQube, Checkmarx & the Secure Pipeline

Security is wired into every pipeline from scaffold time. The DevOps Agent and Security Agent jointly produce a pipeline where security gates are structural, not optional.

What Gets Wired In

- **SonarQube** quality gate enforcement per project.
- **Checkmarx SAST** — full scan on main-branch merges, incremental scans on feature branches. High/Critical block merge; Medium/Low generate prioritized backlog items via Jira MCP.
- **Snyk SCA** for dependency scanning across npm, NuGet, Maven, pip.
- **Trivy** for container image scanning before push to ACR.
- **Cosign** image signing with Kubernetes admission-time verification via Kyverno.
- **Secret scanning** via GitLeaks or TruffleHog on every commit.
- **IaC scanning** (tfsec, Checkov) against Pulumi and Terraform output.

Skills That Back the Pipeline

`sonarqube-adports-quality-gate`, `checkmarx-scan-presets`, `snyk-sca-baseline`, `trivy-container-baseline`, `secret-scanning-remediation`.

Security Posture Reporting

The Portal's **Portfolio Health** view shows per-project SonarQube quality gate status, open Checkmarx findings by severity, open Snyk CVEs, container vulnerability count, and time-to-remediation trends.

Integration with the Jira Workflow

Non-blocking findings (Medium/Low) auto-create Jira tickets tagged `security`. The Ticket Implementation Agent (Section 22) can pick up straightforward security tickets and auto-implement them, subject to review gates.

---

19. Framework Migration Guidelines Engine

Frameworks evolve. Staying current is a security, performance, and talent-retention concern. The Framework Migration Guidelines Engine makes framework migrations a first-class, repeatable capability.

What the Engine Provides

Per-framework migration playbooks. Each covers breaking changes, deprecations, tooling updates, testing priorities, effort and risk bands, AD Ports-specific gotchas.

A Framework Migration Agent that scans a target repo, compares to the preferred version per the framework-lifecycle-policy, pulls the playbook, generates a phased migration plan, executes the migration in a feature branch, runs the full test suite, opens a merge request with a detailed summary, and flags findings requiring human judgment.

Framework lifecycle policy declaring supported versions, deprecation schedule, upgrade triggers, exception process — enforced by the Framework Obsolescence Monitor (Section 23).

Supported Framework Paths

.NET, Angular, Node.js, Postgres, Helm chart API versions, Kubernetes API deprecations — plus additional playbooks added as demand arises.

How Migrations Are Sequenced

Dry-run upgrade → tooling and dependency updates → code modifications (automated codemods plus human-reviewed changes) → test adjustments → configuration updates → full test suite → staging deployment → production rollout plan. Every stage produces versioned artifacts; failures recover from a known-good checkpoint.

---

20. Fleet-Wide Upgrade & Centralized Maintenance Agent

This is one of the highest-leverage capabilities in the whole Portal. Once the Project Registry (Section 26) knows every generated project and its exact stack, a single decision — "upgrade the entire portfolio from .NET 6 to .NET 10" — becomes a coordinated campaign, not 40 independent manual upgrades. Security patches, vulnerability fixes, library updates, and framework migrations across the organization all flow through the same campaign model.

Campaign Concept

A **Fleet Campaign** is a declared upgrade or patch that applies to a set of projects identified by stack fingerprint or explicit inclusion. Every campaign has:

- **A campaign definition.** What is being upgraded (e.g., .NET 6 → .NET 10; Angular 18 → Angular 20; Log4Net 2.0.12 → 2.0.15 to fix CVE-XXX).
- **A scope.** Every project matching a stack query, or an explicit project list. Excluded projects are named with justification.

- **A risk tier.** Low (transparent patch, well-tested codepath) / Medium (minor-version upgrade) / High (major-version upgrade, breaking changes possible) / Critical (zero-day CVE, emergency patch).
- **A playbook.** The migration playbook from the Framework Migration Guidelines Engine, customized for this campaign.
- **An approval chain.** Per risk tier — low tier may need only platform-team approval; high tier requires CAB approval plus per-project tech lead acknowledgment.
- **A rollout plan.** Wave-based rollout — a small pilot wave first (typically 2–3 low-risk projects), then a broad wave, then the remainder. Each wave must succeed before the next begins.
- **A success criterion.** "Upgraded project passes full test suite" as the baseline, optionally extended with performance gates ("p99 latency within 10% of baseline"), stability gates ("no critical Grafana alerts in 24 hours staging soak"), or business gates ("sample BRD scenarios pass integration testing").
- **A rollback plan.** Automatic rollback triggers and manual rollback procedures.

How a Campaign Flows

(1) Campaign proposal. Platform engineer or security officer opens the Portal, navigates to **Fleet Maintenance**, proposes a campaign. The Portal:

- Queries the Project Registry for affected projects.
- Produces impact assessment — which projects, what stack versions, rough effort per project, known risks (is any project past mainstream support on adjacent libraries that might complicate the upgrade?).
- Generates the campaign document (draw.io wave diagram, per-project task list, approval chain, rollback plan) as a reviewable artifact.

(2) Campaign approval. Tech leadership reviews. For high-tier campaigns, CAB approval is required. Every approval is recorded in the Pipeline Ledger. A notification is sent to every affected project's tech lead: "Your project X is in scope for campaign Y. Acknowledge or request exclusion by date Z."

(3) Per-project acknowledgment. Each tech lead sees their project's inclusion in the Portal with estimated effort and scheduled wave. They can acknowledge (default), request exclusion (with justification), or request to join an earlier wave (volunteer to pilot). Ledger records every response.

(4) Per-project agent activation. Once the campaign is approved and the first wave starts, a **per-project Fleet Upgrade Agent** is activated for each project. Each team member of each

project can interact with this agent directly from their IDE or terminal:

> Developer (Cursor): `@adports-fleet what's the status of the .NET 10 campaign on my project?` Agent: "Your project `jul-dgd-declaration` is in wave 2, scheduled for next week. The dry-run upgrade produced 3 breaking changes requiring human review. Want me to pre-open the MR branch so you can look?"
> Developer: `@adports-fleet yes, and flag any issues that block upgrade.`
> Agent: *creates `fleet-upgrade/dotnet10` branch, applies codemods, runs the test suite, opens a draft MR with a summary of changes, a list of 3 items needing human attention, and a link to the campaign's consolidated dashboard.*

The per-project agent uses the same capability fabric, skills, and standards as the master Framework Migration Agent but is scoped to this specific project. Team members do not need to re-do the upgrade themselves — they review what the agent produces.

(5) Automatic verification. For each project in the wave, the Fleet Upgrade Agent:

- Opens a feature branch from the project's latest main commit.
- Applies the campaign's playbook — codemods, dependency bumps, configuration changes, tooling updates.
- Runs the project's full pipeline — build, unit tests, API contract tests, integration tests, Playwright E2E, accessibility scans, k6 load tests, **the Postman integration collection against the deployment from the upgraded branch**. All skills from the QA Automation Agent (Section 16) and Integration Test Agent (Section 17) are reused.
- Produces an **Upgrade Verification Report** covering: build status; unit test pass/fail deltas vs. baseline; Playwright pass/fail deltas; accessibility regressions; load-test performance deltas (p50, p95, p99 latency, error rate, throughput); integration-test environment matrix; any security scan changes.
- Posts the report to the PR as a structured comment and to the campaign dashboard.

(6) Approval and merge. The per-project tech lead reviews the verification report and approves the PR (with standard PR review process). On merge, the campaign dashboard updates the project's status to "Upgraded." If the verification report shows regressions, the tech lead can request remediation from the agent or escalate.

(7) Wave completion and progression. When every project in a wave has either merged or been excused, the next wave begins. Campaigns with failure rates above a declared threshold auto-pause for leadership review.

(8) Rollback. If any project experiences a critical issue post-merge (production incident, Grafana SLO breach), the campaign supports one-click rollback for that project — revert

the merge, redeploy the previous Helm release, restore any schema changes. Rollback is recorded in the ledger.

Why This Matters

Today, a .NET major-version upgrade across 40 AD Ports projects is an 18–24 month effort spanning dozens of teams, each re-learning the migration path independently, with inconsistent timing and coverage. With the Fleet Upgrade Agent, the same upgrade is a campaign with coordinated waves, consistent playbooks, automated verification, and full ledger attribution. The engineering capacity saved compounds every quarter.

For security-driven campaigns (zero-day CVEs), the compression is even more valuable. "Patch Log4Net across every service by end of week" becomes a 48-hour campaign with verified coverage across the portfolio, rather than a multi-week scramble coordinated on Slack.

Per-Team Developer Experience

The key UX principle is that team members **do not re-do the upgrade**. They receive a pre-prepared MR with a verification report, review it, and approve. For simple patches this is literally a one-click merge. For complex upgrades the team reviews the agent's breaking-change flags and either approves the agent's proposed resolution or adjusts.

A tech lead sees a portfolio-level dashboard — their projects' campaign participation, upcoming waves, open upgrade PRs, verification statuses. Engineering leadership sees an organization-level dashboard — campaigns in flight, success rates, projects behind schedule, exclusion requests.

Skills That Back Fleet Upgrades

- **fleet-campaign-policy** — risk-tier definitions, approval chains per tier, rollback rules.
- **fleet-wave-planning** — how to partition projects into waves given dependencies and team capacity.
- **upgrade-verification-report** — what metrics to collect and how to present them.
- **fleet-rollback-playbook** — coordinated rollback procedures across wave members.

---

21. BA & PM Automation — User Story Generation for Jira & Azure DevOps

BAs and PMs spend substantial time translating business requirements into tickets. The BA/PM Story Agent automates the mechanical translation; humans focus on prioritization, scope, and stakeholder alignment.

Inputs: BRD, HLD, meeting notes, existing epic structure, project architectural pattern.

Outputs to Jira or Azure DevOps: Epics (per bounded context), user stories (`As a <role>, I want <goal>, so that <benefit>` with Gherkin acceptance criteria, effort estimate, dependencies, labels, BRD/HLD links), technical tasks for cross-cutting work, test scenarios linked to stories.

Quality safeguards. Every draft story gets a quality score (role clarity, acceptance-criteria measurability, testability, scope appropriateness, traceability). BAs review the draft board, approve or revise, then publish. Re-running on updated BRDs produces diffs, not duplicates.

Handles. `@adports-story analyze`, `@adports-story expand epic X`, `@adports-story refine story Y`.

End-to-end traceability. Every story is tagged with project ID and links to the BRD paragraph that motivated it. The Pipeline Ledger captures the full chain from BRD → story → code → PR → deployment.

---

22. Auto-Implementation from Jira Tickets

Once a project is in active development, the Ticket Implementation Agent can pick up Jira tickets, implement them, run tests, commit, and transition to Done with full commit linkage.

Activation model. Strictly opt-in per project. Requires tech-lead opt-in, declared eligible ticket scope (labels like `auto-eligible`, `small`, `dependency-upgrade`), story-point cap, minimum ticket quality score.

How it works. Webhook trigger → context assembly → plan (commented into ticket) → implement on feature branch → run checks (build, tests, lint, local SAST) → self-correct within bounded iterations → commit with AD Ports-formatted messages → open PR with detailed description → PR Review Agent reviews → ticket transitions to `In Review` → on human approval and merge → ticket to `Done` with commit SHA appended.

What the agent does not do. Approve its own PRs. Make architectural decisions. Modify shared services without separate approval. Touch production directly. Invent tickets.

Eligible patterns. Small CRUD additions, dependency upgrades, security patches, documentation updates, test additions, configuration changes, within-pattern boilerplate.

Observability. Attempt success rate, pickup-to-PR time, review score distribution, merge rate, defects traced to auto-implemented tickets. Auto-pause if metrics deteriorate.

---

23. Continuous Vulnerability & Framework-Obsolescence Radar

The Portal continuously watches the portfolio for security vulnerabilities and framework obsolescence — both actionable rather than informational.

Vulnerability Radar

Scope: every repository plus running container images. Inputs: dependency trees, container SBOMs, public CVE feeds, private advisories.

Scanning: on every commit, every container build, plus a background sweep across every repository at a regular cadence.

Actions: Critical/High with patch → auto-PR proposing upgrade, notification, Fleet Upgrade campaign if portfolio-wide. Critical/High without patch → immediate notification with mitigation guidance and tracking ticket. Medium/Low → weekly digest, backlog tickets.

Dashboards: portfolio view, project view, CVE view.

Framework Obsolescence Monitor

Scope: every framework version across repositories. Source of truth: framework-lifecycle-policy.

Actions: out-of-support version → critical alert + ticket + tech-lead notification. Approaching end-of-support → warning + dry-run migration plan. Past preferred target → informational in portfolio dashboard.

Combined Weekly Digest

Weekly digest to platform leadership: new critical CVEs (and affected projects), projects entering end-of-support, top five remediation backlogs, trend arrows.

Integration with Fleet Campaigns

When the Vulnerability Radar identifies a CVE affecting many projects, it can auto-propose a Fleet Campaign (Section 20) with the campaign scope pre-populated, the playbook drafted, and the risk tier set. Platform engineering reviews and approves. This is how a zero-day CVE becomes an organization-wide coordinated patch within hours.

---

24. Live Health Monitoring of AD Ports Services

Every service generated by the Portal — plus every legacy service registered manually — is subject to **live health monitoring**. The Portal runs continuous health checks, aggregates the results into a dashboard, and fires notifications when services degrade or fail.

Scope

- **Generated services.** Every service registered in the Project Registry is automatically on-boarded to health monitoring.
- **Shared services.** Keycloak, MDM, Notification Hub, Audit, File Service, MPay, CRM, CMS, ERP.
- **External dependencies.** SINTECE, ASYCUDA, SICOEX, Azure regional services — anything the portfolio depends on.
- **Legacy services.** Services not generated by the Portal can be registered manually with their health probe details.

Configurable Health Checks

Each monitored service has a configurable probe set managed through the Portal's **Service Health Admin UI**:

- **Liveness check** — is the service responding at all (HTTP 200 on a liveness path, gRPC HealthCheck, TCP socket open)?
- **Readiness check** — can the service serve real traffic (database reachable, Keycloak reachable, RabbitMQ reachable, required dependencies up)?
- **Business check** — does a sample real business operation succeed (submit a synthetic declaration and confirm the expected response)?
- **Synthetic transaction** — end-to-end Postman-collection-based synthetic, reusing the Integration Test Agent's generated collection (Section 17), run on a schedule against production.

Check intervals are per-check configurable (liveness every 30s, business check every 5 minutes, synthetic every 15 minutes). Timeouts, retry counts, and failure thresholds are all per-check configurable.

Admin UI

The Service Health Admin UI is the primary surface for SREs and on-call engineers:

- **Service inventory.** Every monitored service with current status (green/amber/red), last-check timestamp, uptime percentage (24h / 7d / 30d), recent incident count.
- **Service detail.** Per-service probe configuration, recent health history graph, dependency graph (what other services does this depend on?), linked runbooks, linked Pipeline Ledger entries (what deployments or changes are correlated with recent status changes?).

- **Probe editor.** Add/edit/remove probes, adjust thresholds, test a probe before saving.
- **Escalation editor.** Define notification routes per service — who gets paged for a critical failure, who gets a Slack message for amber status, who gets a daily digest.
- **Silencing.** Temporarily silence a noisy check during planned maintenance.
- **Status page.** An optional public or internal status page driven from the same data — useful for cross-team visibility.

Notification Channels

When a health check fails (or recovers), notifications fire through configured channels:

- Microsoft Teams (default for most AD Ports teams).
- Slack.
- Email.
- PagerDuty for on-call rotation escalation.
- SMS for critical escalations.
- Jira or Azure DevOps for auto-creating incident tickets.
- Webhook for custom downstream consumers.

Notification content is templated and includes the service name, the failed probe, the failure details (HTTP status, response body excerpt, latency), the runbook link, the dependency graph context, and a direct link to the service's dashboard in the Portal.

Severity and Escalation

Each service has a declared severity tier (customer-facing critical, internal critical, important, non-critical). The default escalation chain is derived from severity:

- **Customer-facing critical.** Immediate PagerDuty to primary on-call, 5-minute escalation to secondary, 15-minute escalation to engineering leadership, Jira incident ticket auto-created.
- **Internal critical.** Teams alert to service owner, 15-minute escalation, Jira incident ticket.
- **Important.** Teams alert, daily digest if unresolved.
- **Non-critical.** Daily digest only.

Every escalation step is configurable per service.

Auto-Remediation (Opt-In)

For some classes of failure, the Portal can attempt auto-remediation before escalating to

humans. Opt-in per service and per remediation action:

- **Restart the failing pod** via the AKS MCP if the readiness probe has been red for > N consecutive checks.
- **Clear a saturated cache** if cache-miss rate is abnormally high.
- **Scale up replicas** if CPU or request queue depth exceeds threshold.
- **Failover to secondary region** for services with active-passive regional deployment.

Every auto-remediation action is recorded in the Pipeline Ledger, with the triggering signal, the action taken, the outcome, and a follow-up Jira ticket for review. Auto-remediation does not replace human on-call — it reduces the noise floor and buys time for humans to investigate.

Root-Cause Correlation

When a service goes red, the Portal automatically correlates:

- Recent deployments to this service (from the Pipeline Ledger).
- Recent configuration changes.
- Recent dependency failures (downstream services that are also red).
- Recent fleet-campaign activity affecting this service.
- Recent vulnerability-radar PRs merged to this service.

The correlation is presented in the first notification so the on-call engineer starts investigating with context, not blind. "Service X went red; Deployment Y shipped 8 minutes ago; consider rollback."

Integration with the Incident Post-Mortem Assistant

When an incident is declared (manually or triggered by a critical escalation), the Post-Mortem Assistant (Section 29) pre-populates a post-mortem template with the timeline, the correlated changes, the affected services, and the notification history. The on-call engineer fills in the narrative; the facts are assembled automatically from the ledger and the health history.

Skills That Back Health Monitoring

- **health-check-baseline** — default probe configurations per service archetype.
- **escalation-policy-templates** — severity-tier escalation chains.
- **auto-remediation-playbooks** — safe automated responses to common failure patterns.
- **incident-correlation-queries** — how to join ledger events with health history.

---

25. Pull Request Review Agent & Code Quality Gate

Once a team is building on a Portal-scaffolded project, the Portal stays active as a continuous reviewer.

Every PR triggers the PR Review Agent. It retrieves the project's standards context, runs deterministic checks (linters, Checkmarx, Snyk, SonarQube quality gate, unit test coverage, build, architectural-boundary tests), runs LLM-based review on the diff against coding standards and patterns, scores the PR on a 0–10 rubric (correctness signals, test coverage, architectural alignment, security, documentation), and posts a single consolidated review comment.

Auto-reject gate. Below a configured threshold (default 7/10), merge is blocked. Developer can appeal, address and re-push, or override with audited approver sign-off.

Calibration. Threshold project-configurable. Scores calibrated against human review outcomes on a rolling sample to prevent drift.

Staged rollout. Advisory mode → block on low confidence only → full gate. New projects opt in at the mode they are comfortable with; projects graduate as trust builds.

Every PR review outcome is a Pipeline Ledger event.

---

26. Project Registry & Cross-Project Component Reuse

The Project Registry is the authoritative inventory of every project the Portal has ever touched. It is the directory service that makes fleet operations, cross-project reuse, and portfolio governance possible.

What the Registry Stores

For every project:

- **Identity.** Project ID, display name, parent program (JUL, PCS, Mirsal, internal, etc.), team, business owner.
- **Repository pointers.** GitLab/Azure DevOps URLs for every repo in the project.
- **Stack fingerprint.** Exact versions of every framework, runtime, and major library. .NET version, Angular version, Helm chart API version, Kubernetes target, Postgres version, every NuGet package, every npm package. Refreshed on every merge to main.

- **Integration map.** Which shared services the project consumes (MPay, CRM, CMS, ERP, Keycloak realms, MDM domains). Which external systems it integrates with (SINTECE, ASYCUDA, etc.).
- **Deployment state.** Which environments it runs in, which Helm releases are current, which image digests are deployed.
- **Health status.** Current uptime and incident history from the Service Health Monitor.
- **Compliance scope.** NESA, ISO-27001, UAE-IAR, Angola data-residency, any specific regulatory context.
- **Ownership.** Tech lead, principal architect, security contact, SRE contact.
- **Maintenance coverage.** Opt-ins for Fleet Upgrades, Vulnerability Radar, Ticket Implementation, PR Review enforcement mode.

What the Registry Enables

Fleet operations. "Every project on .NET 6" returns a precise list. "Every project depending on Log4Net below 2.0.15" returns a precise list. Campaigns are scoped by query, not by tribal knowledge.

Cross-project reuse. When a developer is about to build a new component, they can ask: `@adports-registry does any project already implement a Camunda-workflow adapter for SINTECE certificate checks?` The registry searches by stack fingerprint and integration map, returns candidates, and links to the implementing project's ledger entries and code paths. The developer evaluates the reusable component, imports it as a NuGet or npm dependency, or lifts the pattern into a new skill. Reuse becomes discoverable.

Portfolio governance. Executive dashboards — the share of the portfolio on preferred framework versions, the share with passing SonarQube gates, the share with current security posture.

Onboarding. A new engineer can filter projects by team, by stack, by domain, and browse the portfolio at a glance. Attaching the Pipeline Ledger provides instant context on any project.

Registration

New projects are registered automatically at scaffold time. Legacy projects can be registered manually through the Portal UI (provide repo URLs, stack details, ownership) or via a discovery scan that indexes existing GitLab/Azure DevOps organizations and proposes registrations for confirmation.

---

27. Legacy System Migration Mode

Not every project is greenfield. The Portal supports modernization through **Migration Mode**.

The architect uploads legacy documentation plus target standards. The Orchestration Agent runs comparison analysis and produces a side-by-side architecture diagram (current vs. target, components color-coded green/amber/red), a migration plan sequenced by dependency and risk, and per-component artifact bundles (new scaffolded version, data migration scripts, cutover plan, rollback procedures, test validation).

Staged execution. Migration is never all-at-once. Stage 1 (shared services alignment) → Stage 2 (database schema) → Stage 3 (non-critical microservice pilot) → Stage 4 (integrations and middleware) → Stage 5 (critical microservices one-by-one) → Stage 6 (frontend modernization) → Stage 7 (legacy decommissioning). Every stage's artifacts are permanently recallable from the Pipeline Ledger.

---

28. AD Ports Ecosystem Agents (MPay, CRM, CMS, ERP, JUL, PCS)

The biggest accelerator in the whole platform comes from ecosystem agents. Every AD Ports system needs to integrate with some subset of MPay, CRM, CMS, ERP, JUL, PCS, and other internal platforms. Today this integration is re-discovered and re-implemented on every project. Ecosystem agents fix this permanently.

Anatomy of an ecosystem agent. Full specification, instructions, skills, sample code, sandbox credentials path, live MCP server. Each owned by the respective platform team as a first-class product. Versioned; breaking changes trigger required updates.

Priority order. MPay → Keycloak → CRM → Notification Hub → MDM → CMS → ERP → JUL event bus → PCS → SINTECE/ASYCUDA/SICOEX adapters.

Developer experience. `@myadports-mpay — wire a payment settlement flow for my new DGD service` returns the required NuGet package, handler implementation tailored to the project's CQRS pattern, Keycloak client config, webhook controller with idempotency and signature verification, Vault path for the merchant secret, and a working sandbox test.

---

29. Additional Capabilities Worth Adding

Beyond everything already specified, these capabilities would materially strengthen the platform. Each can be explicitly approved or deferred.

Prompt library as a service. Curated, versioned, A/B-tested prompts for common AD Ports tasks. Prompt quality becomes a measured discipline.

Evaluation harness. Golden test projects (real historical BRDs with known-good outcomes) run against every new agent release. Regressions caught before release.

Cost observability per project. LLM tokens, compute, storage, API calls per project. Finance chargeback surface; architects see trade-off signals.

Auto-documentation for existing systems. Point the Portal at any repository; get a Docusaurus site with inferred architecture, API docs, database schema, integration map. Valuable for legacy migration and onboarding.

Self-improving skill library. When the PR Review Agent catches a recurring pattern across projects, the finding surfaces to skill owners with a suggested update. Skills evolve on production evidence.

Chat-with-architecture. Ask free-form questions about any project — "why Camunda over Saga here?", "which services depend on MPay?" — answered from the artifact trail. Institutional memory, searchable.

Synthetic data service. Realistic, compliance-safe test data on demand for any declared schema. Used by QA Automation and Integration Test agents.

Developer productivity analytics. Aggregate metrics — median PR-to-merge time, deploy-to-first-bug time, SLO green rate, platform adoption per team. Portal ROI measurable.

Sovereign / air-gapped operation. On-premises deployment with self-hosted LLMs for ARCCLA or classified workloads.

Scheduled policy reviews. Every instruction has a review cadence; stale instructions flagged in UI.

Release notes generator. Consumes merged PRs, linked Jira tickets, and deployed commits to produce human-readable release notes per service per deployment.

Incident post-mortem assistant. Pre-populates post-mortem templates with timelines, correlated changes, affected services, notification history. On-call engineer fills in the narrative.

Change Advisory Board workflow. First-class CAB support — change requests, CAB meeting agendas, decisions recorded in the ledger, automated follow-through to deployment gates.

Cross-project knowledge graph. Visualize the reference graph from the ledger — which projects consumed context from which, which skills are most reused, which patterns recur. Surfaces candidates for promotion to first-class skills.

Compliance evidence auto-export. For any regulatory audit (NESA, ISO-27001, SOC-2), the Portal assembles a pre-baked evidence package from the ledger with stage approvals, access logs, security scan reports, deployment attestations — cutting audit preparation time dramatically.

Multi-tenant federation. Separate Portal instances per business unit (AD Ports, ARCCLA, MPay), federated so campaigns and shared skills can propagate while respecting tenancy.

---

30. Technology Stack & Build-vs-Integrate Decisions

Build only what creates unique value; integrate proven open-source for everything else. The unique work is the AD Ports-specific fabric (skills, specs, instructions, ecosystem agents, orchestration state machine, PR review rubric, Fleet Upgrade Agent, Project Registry, Pipeline Ledger). Almost everything else is integrated.

Full Technology Stack

Category	Technology	Role	Decision
Portal UI	Angular with PrimeNG	Web UI consistent with JUL	**Build** (thin)
Portal Backend	.NET with CQRS	Portal's own API	**Build** (thin)
Orchestration Workflow	LangGraph	Deterministic agent state machines	Integrate
Multi-agent Patterns	CrewAI	Team-based agent collaboration	Integrate
LLM Gateway	LiteLLM	Provider-agnostic routing + caching	Integrate
LLMs — Judgment	Claude Sonnet + GPT-class	Premium reasoning tasks	Integrate
LLMs — Routine	DeepSeek / Llama	Cheap routine generation	Integrate
Agent Protocols	MCP + A2A	Tool & agent interop	Integrate
Agent Observability	LangSmith + OpenTelemetry	Multi-agent traces	Integrate

Category	Technology	Role	Decision
Durable Workflow	Temporal.io	Long-running orchestrations, fleet campaigns	Integrate
Event Bus	Kafka	Stage transitions, audit events	Integrate
Immutable Event Store	EventStoreDB	Pipeline Ledger backbone	Integrate
Relational DB	PostgreSQL	Projects, users, registry, ledger index	Integrate
Hot Cache	Redis Cluster	Shared project context, session state	Integrate
Object Store	Azure Blob Storage	Artifact cold storage	Integrate
Identity	Keycloak	Portal identity + managed realms	Integrate
Authorization	OpenFGA	Fine-grained Portal permissions	Integrate
Secrets	HashiCorp Vault	Credentials for target systems	Integrate
Policy Engine	OPA (Rego)	Hook Engine evaluation	Integrate
Digital Signatures	Keycloak + standard X.509	Approval signing	Integrate
Health Monitoring	Prometheus + Alertmanager + Blackbox Exporter	Probe execution and routing	Integrate
Synthetic Monitoring	Newman (Postman CLI)	Integration-test synthetics	Integrate
SAST	Checkmarx + Semgrep	PR & pipeline scanning	Integrate
SCA	Snyk + Trivy	Dependency & container scanning	Integrate
Code Quality	SonarQube	Quality gate enforcement	Integrate
Container Platform	AKS	Portal runtime + target	Integrate

Category	Technology	Role	Decision
GitOps	ArgoCD	Continuous delivery	Integrate
IaC	Pulumi + Crossplane	Infra management	Integrate
E2E Testing	Playwright	QA automation surface	Integrate
API Testing	Postman / Newman	Deployed-system verification	Integrate
Contract Testing	Pact	Consumer-driven contracts	Integrate
Load Testing	k6	Performance validation	Integrate
Observability	OpenTelemetry + Prometheus + Grafana + Loki + Tempo	Full-stack telemetry	Integrate
Documentation	Docusaurus	Generated spec sites	Integrate
Diagrams	draw.io (diagrams.net)	Architecture visuals	Integrate
Jira / Azure DevOps	Native APIs via MCP	Work-item management	Integrate
Teams / Slack / PagerDuty	Native APIs via MCP	Notification channels	Integrate
MCP Skills	AD Ports skill bundles	Organizational knowledge	**Build**
MCP Specs	AD Ports spec library	Machine-readable standards	**Build**
MCP Instructions	AD Ports instruction library	Policy-as-text	**Build**
Ecosystem Agents	MPay, CRM, CMS, ERP, JUL, PCS wrappers	One-time investments, huge leverage	**Build**
Orchestrator Logic	Workflow state machine over LangGraph	Where intelligence composes	**Build**
PR Review Rubric	Custom scoring model	Calibrated to AD Ports standards	**Build**

Category	Technology	Role	Decision
Ticket Implementation Agent	Custom bounded-scope implementer	Directly productive work	**Build**
Framework Migration Playbooks	Per-framework guides	AD Ports-specific experience	**Build**
Fleet Upgrade Agent	Campaign orchestration	Portfolio-wide maintenance	**Build**
Project Registry	Fleet inventory with stack fingerprinting	Makes fleet operations possible	**Build**
Pipeline Ledger	Immutable audit + cryptographic chaining	Compliance-grade governance	**Build**
Service Health Monitor	Probe orchestration + UI + notifications	Integrated with ledger and registry	**Build**
Integration Test Agent	Postman-collection generator with BRD-aware payloads	Deployed-system verification	**Build**

Why This Composition

Differentiation sits entirely in the **Build** rows. The UI, MCPs, LLM gateway, observability stack, container platform — all commodity. Value is what we encode into the fabric and how we compose the pieces. That fabric is defensible because only AD Ports has the knowledge to build it.

---

31. Phased Delivery Approach

Four value phases with hard gates. Each phase produces independently valuable capabilities and has an explicit exit criterion.

Phase 1 — Foundation

Goal: Prove the orchestrator can take a BRD, produce a reviewed architecture, and scaffold a single working end-to-end service (Angular MFE + .NET CQRS + Postgres + Keycloak + AKS) with secure pipeline, QA automation, Postman collection, and full ledger recording.

Deliverables: Portal UI, Orchestrator, Backend/Frontend/DevOps/Infrastructure/QA/Integration-Test/Security agents at

foundational capability, five foundational MCPs (Keycloak, GitLab, AKS, SonarQube, Checkmarx), initial skill/spec/instruction fabric, evaluation harness with golden projects, Pipeline Ledger with cryptographic chaining, Project Registry, shared project context over Redis.

Exit criterion — Gate 1: Given a real BRD, Portal produces reviewed approved architecture and scaffolds working code passing build, test, security scan, and initial deployment. Defined percentage of golden projects pass end-to-end. Pipeline Ledger has full recording of every stage with approvals and artifact hashes. Shared project context works across multiple team members.

If the gate fails, Phase 2 does not start.

Phase 2 — Ecosystem & Workflow Integration

Goal: Wire AD Ports ecosystem and developer workflow. Every new project consumes shared services out of the box and is wired to Jira or Azure DevOps. Live health monitoring operational.

Deliverables: Ecosystem agents (MPay, CRM, Notification Hub, MDM, Audit) with full specs, skills, MCPs, sample code, sandbox paths. Database Agent, Integration Agent, BA/PM Story Agent, PR Review Agent (advisory), Hook Engine, IDE integration guides, `adports-ai` CLI GA. **Service Health Monitor** with admin UI, probe execution, notification channels, auto-remediation (opt-in). Vulnerability Radar running. Framework Obsolescence Monitor baseline. Cross-project context referencing.

Exit criterion: Defined pilot project count onboarded. PR review agent achieves high human-agreement. BA story quality acceptable. Service Health Monitor covers full pilot-project set with active notifications.

Phase 3 — Intelligence, Governance & Fleet Operations

Goal: Platform becomes the default path. Governance matures. **Fleet operations active.**

Deliverables: PR Review Agent with auto-reject gate (opt-in). Ticket Implementation Agent in bounded pilot. Framework Migration Agent with major-framework playbooks. Migration Mode. Additional ecosystem agents (CMS, ERP, JUL event bus, PCS, SINTECE, ASYCUDA, SICOEX). **Fleet Upgrade Agent** with first production campaigns (security patches, then minor framework upgrades). Prompt library, evaluation harness expansion, self-improving skills. Auto-documentation for existing repos. Cost observability. Chat-with-architecture. Vulnerability Radar producing auto-PRs. Fabric at critical mass.

Exit criterion: Majority of new projects start via Portal. Median bootstrap time substantially reduced. Ticket Implementation Agent acceptable PR merge rate. Framework migration cycle time reduced. **First successful fleet campaign completed end-to-end across multiple projects.**

Phase 4 — Scale & Sovereignty

Goal: Production maturity, enterprise audit readiness, sovereign cloud option, advanced governance.

Deliverables: Sovereign / air-gapped deployment playbook with self-hosted LLM. SOC-2 Type I readiness. NESA and UAE-IAR compliance package for Portal's own operations. Synthetic data service. Scheduled policy reviews. Developer productivity analytics. Release notes generator. Incident post-mortem assistant. CAB workflow. Federated Portal topology for ARCCLA. Compliance evidence auto-export.

Exit criterion: External audit passed. Most new projects and meaningful share of legacy migrations on Portal. Fleet campaigns running at portfolio scale. Platform operates with sustainable run-mode resourcing.

---

32. Risk Analysis & Mitigation

Risk	Severity	Prob	Mitigation
Gate 1 failure — orchestrator cannot produce reliable scaffolds	Critical	Medium	Walking skeleton early. Golden-project harness from day one. Hard go/no-go. Template-first architecture keeps LLM surface small.
Organizational adoption resistance	High	High	Early-adopter program. Measurable wins published. Portal does not force migration — optional first, default later.
Standards library rots	High	High	Skills/specs/instructions owned by platform teams as products. Quarterly review cadence enforced by tooling. PR Review and Vulnerability Radar feed back into skill updates.
Ecosystem agents drift from real systems	High	Medium	Each agent owned by respective platform team with semver contract. Automated smoke tests against sandbox environments.
LLM cost blowout	Medium	High	Five-strategy cost optimization. Per-project budget hooks. Self-hosted model option in Phase 4.
Generated code quality below human standard	High	Medium	Template-first. Build and test gates. PR Review calibrated. Advisory before enforcing.

Risk	Severity	Prob	Mitigation
Security vulnerabilities in generated code	Critical	Medium	Mandatory SAST and SCA. Security Agent in every orchestration. SonarQube quality gate blocking.
Ticket Implementation Agent produces low-quality PRs	High	Medium	Opt-in. Bounded scope. PR Review gate. Auto-pause on metric degradation. Humans always merge.
Fleet campaign breaks multiple projects simultaneously	Critical	Medium	Wave-based rollout with pilot wave. Auto-pause if failure rate exceeds threshold. Per-project verification report gate. One-click rollback. CAB approval for high-risk campaigns.
Service Health Monitor produces alert fatigue	High	Medium	Severity-tier based routing. Auto-remediation for low-severity. Silencing for maintenance. Calibration against human triage outcomes.
Pipeline Ledger storage growth	Medium	High	Tiered storage (Redis hot, EventStore warm, Blob cold). Index compaction. Retention policies per compliance class.
Shared project context leaks sensitive data between teams	Critical	Medium	Scrubbing at capture time. OpenFGA-based cross-project access control. Audit every cross-project context reference.
Vendor lock-in to specific LLM provider	Medium	Medium	LLM gateway abstraction. Multi-provider routing. Self-hosted fallback.
Legacy migration regressions	High	Medium	Component-by-component execution. Full artifact trail per stage. Validation gates. Rollback in every plan.
Framework migration breaks production	High	Medium	Dry-run first. Full test suite gate. Staging soak. Explicit rollout plan. Human review of judgment-required items.
Data residency violations via LLM egress	Critical	Low	Redaction hooks pre-LLM. Classified projects routed to on-premises LLM. Data-residency declaration enforced.
BA story generation vague or duplicate	Medium	Medium	Story quality score. BA review before publish. Idempotent re-generation produces diffs.

Risk	Severity	Prob	Mitigation
Scope creep into a build-everything platform	High	Medium	Strict build-vs-integrate discipline. Monthly scope reviews. Explicit ban on custom DSL or runtime.
Pipeline Ledger tampered or disputed	Critical	Low	Cryptographic chaining. Periodic notarization to trusted timestamping. Immutable object store with legal-hold. Verify-on-read tooling.

33. Success Metrics & KPIs

KPI	Direction	What It Measures
Median project bootstrap time	↓	Calendar time from BRD upload to first working scaffold
% of new projects starting via Portal	↑	Adoption across engineering
Orchestrator first-attempt success rate	↑	Golden-project pass rate per release
PR Review Agent agreement with humans	↑	Calibration of review rubric
False-reject rate on PR gate	↓	Confidence in auto-rejects
Median time from BRD to approved architecture	↓	Orchestrator throughput
Ecosystem agents available	↑	Portfolio coverage of shared systems
Skills / specs / instructions in library	↑	Fabric depth
Ticket Implementation Agent PR merge rate	↑	Auto-implementation quality
Portfolio mean time to CVE remediation	↓	Security posture
% of portfolio on preferred framework versions	↑	Currency health
QA automation coverage of acceptance criteria	↑	Test suite completeness
Integration-test pass rate across environments	↑	Deployed-system health

KPI	Direction	What It Measures
Fleet campaign success rate (projects upgraded / projects in scope)	↑	Fleet-wide maintenance effectiveness
Median time from security-patch-ready to 100% fleet coverage	↓	Emergency-response agility
Service Health Monitor MTTR	↓	Operational responsiveness
Service Health Monitor alert-to-real-incident ratio	↓	Alert-fatigue avoidance
Pipeline Ledger queries per week	↑	Governance surface adoption
Compliance evidence preparation time	↓	Audit efficiency
Legacy components migrated via Portal	↑	Modernization throughput
Stories from BA Agent accepted without rewrite	↑	BA automation quality
Cross-project component reuse rate	↑	Registry and context leverage
Developer NPS for Portal experience	↑	Actual-user satisfaction

Targets per phase are set during phase planning against baselines measured in Phase 1.

---

34. Conclusion & Next Steps

The AI Portal is a pragmatic, high-leverage investment that turns AD Ports' accumulated organizational knowledge into executable fabric — consumable by the Portal's orchestrator and by any IDE or AI tool the developers already use. Unlike ambitious platform rewrites, this proposal builds on the existing stack (Keycloak, AKS, .NET, Angular, GitLab, Azure DevOps), respects existing tooling (Copilot, Cursor, Claude Code), and focuses its novelty budget on what is uniquely AD Ports: the **standards, workflows, ecosystem integrations, portfolio-scale maintenance, and institutional memory made programmatically accessible**.

What makes this more than a scaffolding tool is the breadth of automation across the full lifecycle. Security scanning, QA automation, deployed-system verification, BA story generation, Jira ticket implementation, vulnerability radar, framework migration, fleet-wide upgrades, live service health monitoring, and immutable audit governance all live in the same fabric as the scaffolding. The same skills that teach the Backend Agent how to

write a CQRS handler teach the Ticket Implementation Agent how to add one, teach the PR Review Agent how to grade one, teach the Framework Migration Agent how to upgrade one when the runtime changes, and teach the Fleet Upgrade Agent how to roll that upgrade across the portfolio. One standards library, many consumers, compound leverage that accrues for the entire lifecycle of every project.

The risks are real but manageable. Gate 1 is the single most important checkpoint. If the orchestrator cannot reliably produce standards-compliant scaffolds by end of Phase 1, the architecture needs rework before expansion.

Immediate Next Steps

Action	Owner
Review this draft with CTO, Heads of Engineering, principal architects. Agree on scope boundaries.	Tech Leadership
Finalize Phase 1 team composition and squad structure.	Engineering Leadership
Identify early-adopter internal projects for Phase 1 pilot.	Product + Tech Leadership
Stand up baseline infrastructure — AKS, Postgres, Redis cluster, LangSmith, LLM gateway, EventStoreDB.	Infra Team
Walking-skeleton sprint — Portal UI endpoint → Orchestrator stub → Backend Agent stub → generated repo with ledger entry.	Full Team
Author initial fabric — minimum viable skill/spec/instruction set.	Fabric Squad + Principal Architects
Establish golden-project harness with real historical BRDs and known-good outcomes.	Orchestrator + Fabric Squads
Design and prototype the Pipeline Ledger with cryptographic chaining.	Governance Squad
Design and prototype shared project context in Redis with permissions.	Core Squad
First end-to-end demo — real BRD produces approved architecture and deployed scaffold with full ledger trail.	Full Team
Gate 1 review — go/no-go for Phase 2.	All Stakeholders

Questions for the Review

- **Primary IDE target.** Cursor, Copilot (VS Code), or Claude Code first? Suggest Copilot + Cursor given current AD Ports distribution.
- **Primary LLM provider for Phase 1.** Anthropic, Azure OpenAI, or Bedrock? Suggest Anthropic primary + Azure OpenAI fallback via LLM gateway.
- **Primary work-item platform.** Jira only, Azure DevOps only, or both from day one? Suggest both.
- **Which ecosystem agent is Phase 2 priority #1?** Suggest Keycloak (universal dependency) + MPay (highest developer pain).
- **PR Review enforcement rollout.** Suggest opt-in in Phase 2, default-on in Phase 3.
- **Ticket Implementation rollout.** Strongly suggest opt-in indefinitely. This capability is powerful and should never be the default.
- **Fleet Upgrade Agent rollout.** Suggest starting Phase 3 with security-patch campaigns (low-risk, high-value) before framework campaigns.
- **Service Health Monitor scope at Phase 2 launch.** Generated services only, or also register legacy services immediately? Suggest generated-services-only at Phase 2 launch, legacy in Phase 3 once patterns are proven.
- **Pipeline Ledger notarization.** Internal timestamping service, external timestamping service, or no external notarization? Suggest internal for Phase 2, external for Phase 4 when compliance scope demands it.
- **Cross-project context sharing.** Default-open within a program (e.g., JUL), default-closed across programs? Suggest default-closed everywhere, explicit permission grants for reuse.
- **Sovereign cloud scope.** Is a self-hosted LLM required for any Phase 1 or Phase 2 project? If yes, Phase 4 capabilities move earlier.